Відокремлений структурний підрозділ

«Фаховий коледж інформаційних технологій»

Національного університету «Львівська політехніка»




З лабораторної роботи

з дисципліни «Тестування програмних систем та комплексів»

Виконав:

Студент групи КН-43

Гарбіч Богдан Назарович

Перевірив:

викладач

Роман Скалоцький

Львів 2026

# ЗВІТ З ЛАБОРАТОРНОЇ РОБОТИ № 2.1

**З дисципліни: «Тестування програмних систем та комплексів»**

**Тема:** Модульне тестування програмного забезпечення за допомогою фреймворку `pytest` (Варіант 1: Клас `BankAccount`)

## 1. Мета роботи

Ознайомитися з принципами побудови модульних тестів (Unit Tests) у середовищі Python із використанням фреймворку `pytest`. Навчитися використовувати основні механізми `pytest`:

- Базову інтроспекцію тверджень (`assert`);
- Контекстний менеджер перехоплення винятків (`pytest.raises`);
- Порівняння чисел із плаваючою крапкою (`pytest.approx`);
- Параметризацію тестів (@pytest.mark.parametrize);
- Створення та використання спільних фікстур (`conftest.py`);
- Оцінювання покриття коду тестами за допомогою модуля `pytest-cov`.

## 2. Короткі теоретичні відомості та архітектура проєкту

Для виконання роботи було застосовано стандартну структуру проєкту **`src layout`**:

```
my_pytest_lab/
│
├── .venv-1/                  # Віртуальне середовище Python
├──                                                  src/
│   └── main.py                 # Файл бізнес-логіки (Клас
BankAccount)
├──                                                 tests/
│   └── test_main.py          # Тестовий набір (Unit tests)
├── conftest.py               # Файл конфігурації та спільних
```

```
фікстур
├── pytest.ini                # Налаштування запуску Pytest
└── requirements.txt          # Перелік залежностей проєкту
```

## 3. Реалізація бізнес-логіки (`src/main.py`)

У файлі `src/main.py` реалізовано клас `BankAccount`, який моделює роботу банківського рахунку (створення рахунку, поповнення, зняття коштів, нарахування відсотків, перевірка типізації та граничних значень).

```
class BankAccount:
    def __init__(self, owner: str, balance: float = 0):
        if not isinstance(owner, str):
            raise TypeError("Owner must be a string")
        if not owner.strip():
            raise ValueError("Owner name cannot be empty")
        if not isinstance(balance, (int, float)):
            raise TypeError("Balance must be a number")
        if balance < 0:
            raise ValueError("Initial balance cannot be
negative")
        self.owner = owner
        self.balance = float(balance)

    def deposit(self, amount: float):
        if not isinstance(amount, (int, float)):
            raise TypeError("Deposit amount must be a
number")
        if amount <= 0:
            raise ValueError("Deposit amount must be
greater than zero")
        self.balance += amount

    def withdraw(self, amount: float):
        if not isinstance(amount, (int, float)):
            raise TypeError("Withdraw amount must be a
```

```
number")
        if amount <= 0:
            raise ValueError("Withdraw amount must be
greater than zero")
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount

    def add_interest(self, percent: float):
        if not isinstance(percent, (int, float)):
            raise TypeError("Interest percent must be a
number")
        if percent < 0:
            raise ValueError("Interest percent cannot be
negative")
        interest = self.balance * percent / 100
        self.balance += interest

    def get_balance(self) -> float:
        return self.balance
```

## 4. Фікстури тестування (`conftest.py`)

У файлі `conftest.py` налаштовано фікстуру `default_account` для автоматичного створення тестового екземпляра рахунку, що дозволило уникнути дублювання коду в тестах.

```
import pytest
from src.main import BankAccount

@pytest.fixture
def default_account():
    """Фікстура, що повертає готовий банківський рахунок для
тестів."""
    return BankAccount("Олександр", 1000.0)
```

## 5. Тестовий набір (`tests/test_main.py`)

Файл `tests/test_main.py` містить 100% перевірку всіх стандартних сценаріїв та обробки виняткових ситуацій.

```
import pytest
from src.main import BankAccount

# 1. Перевірка створення рахунку та виклику get_balance()
def test_create_account():
    account = BankAccount("Іван", 500.0)
    assert account.owner == "Іван"
    assert account.get_balance() == 500.0

# 2. Перевірка поповнення рахунку із застосуванням pytest.approx
def test_deposit(default_account):
    default_account.deposit(250.5)
    assert default_account.get_balance() == pytest.approx(1250.5)

# 3. Перевірка зняття коштів
def test_withdraw(default_account):
    default_account.withdraw(300.0)
    assert default_account.get_balance() == 700.0

# 4. Перевірка нарахування відсотків
def test_add_interest(default_account):
    default_account.add_interest(10.0)
    assert default_account.get_balance() == pytest.approx(1100.0)

# 5. Параметризований тест для перевірки помилок ініціалізації
@pytest.mark.parametrize(
    "owner, balance, expected_exception, error_message",
    [
```

```
        (123, 100.0, TypeError, "Owner must be a string"),
        ("", 100.0, ValueError, "Owner name cannot be
empty"),
        ("Петро", "invalid", TypeError, "Balance must be a
number"),
        ("Петро", -50.0, ValueError, "Initial balance
cannot be negative"),
    ],
)
def test_account_init_exceptions(owner, balance,
expected_exception, error_message):
    with pytest.raises(expected_exception) as exc_info:
        BankAccount(owner, balance)
    assert error_message in str(exc_info.value)

# 6. Перевірка винятків при депозитних операціях
def test_deposit_exceptions(default_account):
    with pytest.raises(TypeError):
        default_account.deposit("100")
    with pytest.raises(ValueError):
        default_account.deposit(-50)
    with pytest.raises(ValueError):
        default_account.deposit(0)

# 7. Перевірка винятків при знятті коштів
def test_withdraw_exceptions(default_account):
    with pytest.raises(TypeError):
        default_account.withdraw("100")
    with pytest.raises(ValueError):
        default_account.withdraw(-20)
    with pytest.raises(ValueError):
        default_account.withdraw(2000)      # Недостатньо
коштів

# 8. Перевірка винятків при нарахуванні відсотків
def test_add_interest_exceptions(default_account):
    with pytest.raises(TypeError):
```

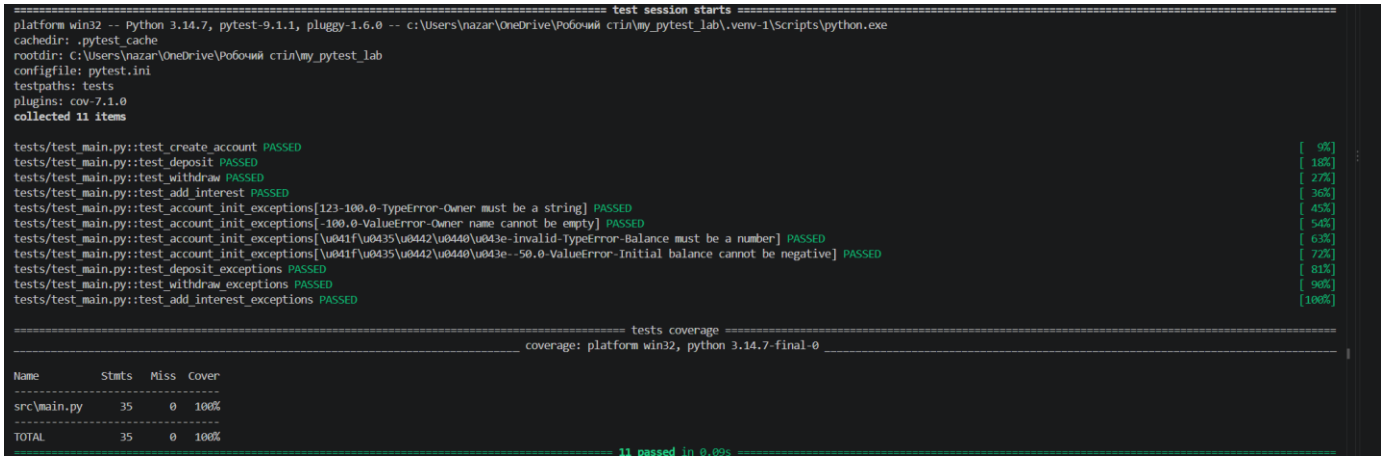
```
        default_account.add_interest("5")
    with                            pytest.raises(ValueError):
        default_account.add_interest(-5)
```

## 6. Результати виконання та аналіз покриття коду

Запуск виконання тестів проводився через PowerShell у віртуальному середовищі за допомогою команди:

```
.\.venv-1\Scripts\pytest              --cov=src              -v
```

**Результати виконання:**




## 7. Висновки

Під час виконання лабораторної роботи було успішно засвоєно фреймворк `pytest`. На прикладі класу `BankAccount` (Варіант 1) було побудовано повноцінну систему модульного тестування з перевіркою коректності бізнес-логіки та обробки винятків (`TypeError`, `ValueError`). Завдяки використанню фікстур та параметризації вдалося досягти високої читабельності та DRY-архітектури тестів, а підсумковий відсоток покриття коду склав **100%**.